ROOTCAUSE

F-DBTOOL — workable-items edit→sync round-trip desync — root cause + fix

Revision: 1 **Last modified:** 2026-07-12T03:45:00Z

Zero-risk statement

All investigation and validation below was performed exclusively against **copies** of `docs/workable_items.db` under `/tmp/wifix_work/`. The live `docs/workable_items.db`, `docs/Issues.md`, and `docs/Fixed.md` were never opened for writing; their checksums are unchanged from session start (`docs/workable_items.db=66da65b31dc629ae9d8fbbce45aa1960`, `docs/Issues.md=9f2c6a4a34a37b7ba032cc30245306a2`, `docs/Fixed.md=c9e37c9931a700d90146c1b3cc585273`). No `git add` / `git commit` was run. The fix below is landed **only** in the working tree of the `constitution` submodule (uncommitted), touching exactly four source files.

1. Root cause

Primary defect (data corruption): every mutating subcommand that reads an item via `loadItem(db, id, location)` and then writes it back with `UPDATE items SET ... WHERE atm_id=? AND current_location=?` — i.e. `update`, `block`, `reopen`, and `obsolete-details` — was blind to the schema's true 3-tuple PRIMARY KEY `(atm_id, current_location, representation)` (GAP A / §11.4.90 dual-representation support, added so the SAME ticket can exist both as a pipe-table closure row (`representation='table'`) and a full H2 narrative section (`representation='section'`) in the same tracker — the HXC-044 shape). `loadItem` never even populated `it.Representation` (Go zero-value `""`, which `repOrDefault()` silently reports as `"section"` regardless of which row was actually read), and its `SELECT` had no `ORDER BY`, so on a dual-representation item like HXC-044 it returned *whichever* row SQLite scanned first — nondeterministic. The subsequent `UPDATE ... WHERE atm_id=? AND current_location=?` (also missing `representation`) then matched **both** rows and overwrote **both** with the same `body_md` — clobbering the sibling representation's content.

`constitution/scripts/workable-items/cmd/workable-items/crud.go:368` (pre-fix `loadItem`) and the four call-sites' `UPDATE` statements in `mutate.go` (`updateCmd` ~L196, `reopenCmd` ~L332, `blockCmd` ~L449) and `obsolete.go` (`obsoleteDetailsCmd` ~L132) are the exact locations.

Amplifying defect #1 (idempotency bug, `obsolete.go:injectObsoleteDetails`, pre-fix ~L156-189): the function dropped a pre-existing `**Obsolete-Details:**` line and, in the *same* pass, scanned for the insertion point by looking one line ahead (`lines[i+1]`) to decide whether the current meta line was the *last* contiguous meta field. When a body already carried an Obsolete-Details line (the normal, "this operation should be idempotent" case), that stale line was itself shaped like a generic `**Field:** value` meta line, so the lookahead saw it and concluded the *real* last field ("`**Assigned-To:**`") was **not** the last one — the loop never found an insertion point and fell through to the "append at the very end" fallback. Because `strings.Join` never adds a trailing separator, the new Obsolete-Details line landed as the body's **last byte, with no trailing newline** — right after the prose paragraph instead of in its correct position inside the heading-adjacent meta block.

Amplifying defect #2 (`db.go:renderDocument`'s `appendSegment`, pre-fix ~L654-659): the glue-prevention logic that inserts a separating `\n` when the accumulated buffer doesn't end in one only fired when the **next** segment's content started with `"## "` (a heading). A newline-less body followed by anything else — in particular a pipe-**table** row (starts `"| "`) — was concatenated with **zero separation**. Once defect #1 left HXC-044's `section`-representation body newline-less, and the missing- representation-scope defect (primary) had *also* copied that exact newline-less body onto HXC-044's `table`-representation row, the very next document segment (the next chronologically-closed item's pipe row) glued directly onto HXC-044's tail. From that byte onward `parseFixed` no longer saw line-start `##/|` boundaries, so it silently absorbed **every subsequent item in Fixed.md** into HXC-044's parsed body — reported by `diff` as those ~188 items being "present in DB, absent in Markdown", plus HXC-044 itself showing `body differs (md=99497 bytes db=656 bytes)` — the exact shape described in the task ("md body=33796B vs db body_md=663B").

2. Minimal reproduction (on a COPY; `/tmp/wifix_work/copyA.db` = untouched

DB copy, `copyC.db` = mutated copy)

```
$ /tmp/wi_fix sync db-to-md --db copyA.db --out-issues IssuesA.md --out-fi
$ /tmp/wi_fix diff --db copyA.db --issues IssuesA.md --fixed FixedA.md
diff: DB and Markdown are in sync          # untouched DB round-trips clea
```

```
$ cp copyA.db copyC.db
$ /tmp/wi_fix obsolete-details HXC-044 --db copyC.db \
    --since 2026-07-12 --reason not-reproducible --superseding none \
    --evidence evidence/HXC-044.txt
$ sqlite3 copyC.db "SELECT atm_id, representation, length(body_md) FROM it
HXC-044|section|652
HXC-044|table|652                          # <-- BOTH rows now byte-identi

$ /tmp/wi_fix sync db-to-md --db copyC.db --out-issues IssuesC.md --out-fi
$ /tmp/wi_fix diff --db copyC.db --issues IssuesC.md --fixed FixedC.md
~ HXC-044 body differs (md=99497 bytes db=656 bytes)
~ HXC-044 body differs (md=657 bytes db=656 bytes)
- FIX-2026-05-19 present in DB, absent in Markdown
... (186 more "present in DB, absent in Markdown" lines) ...
diff: 190 difference(s)
```

`update --id HXC-044 --location Fixed --severity High` on the same untouched copy reproduces an equivalent 189-difference desync via the identical missing-`representation`-scope defect (no idempotency-bug interaction, so a smaller — but still severe — blast radius). `close` was also tested (on `HXC-122`, a non-dual-representation item) and, as expected, does **not** reproduce the corruption by itself: the corruption requires a **dual-representation item** as the mutation target, and HXC-044 is currently the *only* item in the live DB carrying both representations (verified: `SELECT representation, count(*) FROM items GROUP BY representation` → `section|136`, `table|188`; only HXC-044 has both for the same id+location).

3. Fix — validated, landed in the working tree (uncommitted)

Files touched (all inside `constitution/scripts/workable-items/cmd/workable-items/`):

- **`crud.go`** — `loadItem`: SELECT + Scan the row's real `representation`; `ORDER BY CASE WHEN representation='section' THEN 0 ELSE 1 END LIMIT 1` so a dual-representation item deterministically yields its `section` row (matching every existing caller's semantic intent — narrative-content mutation) while a single-representation item (the overwhelming majority — every closure-only pipe row) is returned unchanged. `closeCmd`'s Issues-side `DELETE` is now scoped `AND representation=?`, and its Fixed-side `INSERT` now carries the source row's own `representation` instead of relying on the schema's implicit `'section'` default (closes the same defect class on the

Issues→Fixed move path; currently dormant — no Issues-side dual-rep item exists today — but symmetric with the confirmed defect).

- **`mutate.go`** — `updateCmd`, `reopenCmd`, `blockCmd`: each `UPDATE`'s `WHERE` clause now includes `AND representation=?` bound to the loaded item's `repOrDefault()`, so the write only ever touches the row that was actually read.
- **`obsolete.go`** — `obsoleteDetailsCmd`'s `UPDATE` scoped by representation identically; **and** `injectObsoleteDetails` rewritten to drop stale `**Obsolete-Details:**` line(s) in a first pass and run the insertion-point scan over the *cleaned* list, so the lookahead can never mistake a soon-to-be-removed line for a genuine blocking meta field. Idempotent re-runs now correctly replace the line in place (verified — see §4).
- **`db.go`** — `renderDocument`'s `appendSegment` glue-prevention generalised from "insert a separator only before a heading" to "insert a separator whenever the buffer doesn't already end in a newline" — closes the *whole* glue-defect class (not just the heading instance) as defense-in-depth, so a future body-generation bug that leaves a body newline-less can corrupt at most that one item's parse, never cascade through the rest of the document. Proven a strict no-op for every well-formed body: the full existing test suite (which includes several byte-identical round-trip fixtures) stays green.

All four fixes are minimal (each is a `WHERE`-clause scope addition, a `SELECT`/`ORDER BY` addition, an insertion-order correction, or a one-line condition generalisation) and backward-compatible: every single-representation item (the vast majority of the DB) is read/written identically to before.

4. Validation evidence (captured this session, on copies only)

```
$ cd constitution/scripts/workable-items && go build -o /tmp/wi_fix ./cmd/
$ go vet ./...
$ go test -count=3 ./...

# Re-run the EXACT original repro against the FIXED binary:
$ cp copyA.db copyC2.db
$ /tmp/wi_fix obsolete-details HXC-044 --db copyC2.db --since 2026-07-12 \
    --reason not-reproducible --superseding none --evidence evidence/HXC-0
$ sqlite3 copyC2.db "SELECT atm_id, representation, length(body_md) FROM i
HXC-044|section|652
HXC-044|table|516                          # <-- sibling representation UN
$ /tmp/wi_fix sync db-to-md --db copyC2.db --out-issues IssuesC2.md --out-
$ /tmp/wi_fix diff --db copyC2.db --issues IssuesC2.md --fixed FixedC2.md
diff: DB and Markdown are in sync          # 0 differences (was 190 pre-fi
```

```
# update --severity on the same dual-rep item:
$ cp copyA.db copyD2.db
$ /tmp/wi_fix update --db copyD2.db --id HXC-044 --location Fixed --severi
$ /tmp/wi_fix sync db-to-md --db copyD2.db --out-issues IssuesD2.md --out-
$ /tmp/wi_fix diff --db copyD2.db --issues IssuesD2.md --fixed FixedD2.md
diff: DB and Markdown are in sync          # 0 differences (was 189 pre-fi

# Idempotent re-run of obsolete-details on the already-obsoleted item (2nd
$ cp copyC2.db copyC3.db
$ /tmp/wi_fix obsolete-details HXC-044 --db copyC3.db --since 2026-07-13 \
    --reason duplicate-of --superseding HXC-999 --evidence evidence/HXC-04
$ /tmp/wi_fix sync db-to-md --db copyC3.db --out-issues IssuesC3.md --out-
$ /tmp/wi_fix diff --db copyC3.db --issues IssuesC3.md --fixed FixedC3.md
diff: DB and Markdown are in sync          # correctly replaces the Obsole

# validate on both mutated copies:
$ /tmp/wi_fix validate --db copyC2.db | tail -1
validate: OK — 324 items, all invariants satisfied
$ /tmp/wi_fix validate --db copyE.db | tail -1     # copyE = close HXC-125
validate: OK — 324 items, all invariants satisfied

# Untouched-DB baseline (still clean after the fix, proving no regression)
$ /tmp/wi_fix sync db-to-md --db copyA.db --out-issues IssuesA.md --out-fi
$ /tmp/wi_fix diff --db copyA.db --issues IssuesA.md --fixed FixedA.md
diff: DB and Markdown are in sync
```

5. Related-but-out-of-scope findings (not fixed in this pass)

- `mutate.go:reopenCmd`'s `doc_segments` relocation (`removeItemSegment` / `appendSegment`) is still representation-blind (deletes/re-adds without a `representation` filter/column). This is the *segment* analogue of the fixed *items*-table bug, but it is currently **dormant**: no item that is both dual-representation and eligible for `reopen` exists in the live tree (HXC-044, the only dual-rep item, carries a terminal `Obsolete` status). Flagging per §11.4.124 (investigate-before-remove/fix; not silently patched here to keep this change minimal and precisely scoped to the confirmed, reproduced defect).
- `crud.go:closeCmd`'s `renderItemBody(...)`-based full body regeneration (discarding any body content beyond title/type/severity/description) is the same *class* of truncation bug already fixed for `update` (the "SPK-481" comment in `mutate.go`) but was never extended to `close`. It does **not** cause a `diff` desync (both DB and freshly-regenerated Markdown get the same

regenerated body), so it is orthogonal to the reported defect — a genuine data-loss risk, but a separate ticket.
- `group.go`'s `group set` command has the same unscoped-by-representation `UPDATE` pattern, but it only ever writes `logic_group`/`destination` (classification metadata), not `body_md` — cross-representation duplication there is comparatively low-severity and was left untouched.

6. Summary

Root cause: `constitution/scripts/workable-items/cmd/workable-items/crud.go` (`loadItem`) plus the four write-paths in `mutate.go` (`update`/`reopen`/`block`) and `obsolete.go` (`obsolete-details`) were blind to the schema's 3-tuple primary key's `representation` component, so on the DB's one dual-representation item (HXC-044) a write clobbered both its `section` and `table` rows with the same content; a second, independent bug in `obsolete.go`'s `injectObsoleteDetails` then left that clobbered body newline-less, which a third gap (the heading-only glue guard in `db.go`:`renderDocument`) failed to separate from the next document segment — cascading the corruption into an unparseable tail that swallowed ~188 subsequent items, producing exactly the 176-190-difference `diff` explosion reported.

Fix status: VALIDATED. All four files are fixed in the `constitution` submodule's working tree (uncommitted, per instructions). `go build`, `go vet`, and `go test -count=3 ./...` all pass. The exact original reproduction (`obsolete-details HXC-044` and `update --severity HXC-044` on fresh copies of the live DB) now produces `diff: DB and Markdown are in sync` (0 differences) where it previously produced 190 / 189 differences, and the untouched-DB baseline round-trip remains clean (no regression).